Software Requirements Specification

(SRS)

AI Code Security Analyzer

Course Code: CSE4104
Section: 7C
Team Name: CSE4104-7C-T04

Team Members

No.	Name	Student ID
1	Sadia Islam Jhumur (Team Leader)	11230121171
2	Sakil Hasan Saikot	11230121188
3	Tanjilanta Kanchi	11220121034
4	Farin Islam Mim	11230121061

Department of Computer Science and Engineering
Northern University of Business and Technology, Khulna
2026

1 Introduction

1.1 Project Title

AI Code Security Analyzer

1.2 Project Overview

AI Code Security Analyzer is an AI-powered Software-as-a-Service (SaaS) platform designed to analyze source code and identify security vulnerabilities before software deployment. The system allows users to upload source code files, paste source code directly, or submit GitHub repositories for automated security analysis.

The platform utilizes advanced Artificial Intelligence models such as Claude, GPT-4o, Gemini, and DeepSeek to perform semantic code analysis and generate detailed vulnerability reports. Unlike traditional rule-based scanners, the system understands the context and logic of source code, enabling more accurate detection of security flaws and insecure coding practices.

1.3 Problem Statement

Modern software development is becoming increasingly complex, while security vulnerabilities continue to be one of the leading causes of cyberattacks, data breaches, ransomware incidents, and unauthorized system access.

Traditional security analysis tools primarily rely on predefined rules and pattern matching techniques. Although effective in detecting known vulnerabilities, these tools often fail to identify logic-level vulnerabilities, generate excessive false positives, and require complex configuration.

Developers, students, startups, and small organizations often lack access to affordable and intelligent security auditing tools capable of identifying vulnerabilities before deployment. As a result, insecure software is frequently released into production environments.

The proposed AI Code Security Analyzer addresses these challenges by leveraging advanced AI models to automatically analyze source code, detect vulnerabilities, provide contextual explanations, and recommend remediation strategies.

1.4 Objectives

- Develop an AI-powered platform for automated code security analysis.
- Detect common and advanced software vulnerabilities.
- Reduce dependence on manual security reviews.
- Generate comprehensive vulnerability reports.

- Provide AI-generated remediation recommendations.
- Support GitHub repository analysis.
- Improve software security awareness among developers and students.
- Create a scalable SaaS platform for secure software development.

1.5 Scope of the Project

The system will:

- Allow users to register and log in.
- Allow source code upload and direct code input.
- Accept GitHub repository URLs.
- Perform AI-powered vulnerability analysis.
- Generate security reports.
- Store scan history.
- Classify vulnerabilities based on severity levels.
- Provide remediation suggestions.
- Export reports in multiple formats.

The system will not:

- Execute uploaded source code.
- Automatically modify user source code.
- Replace professional penetration testing services.

2 Functional Requirements

FR-01: Users can register using email and password.

FR-02: Users can securely log into the platform.

FR-03: Users can manage their profiles.

FR-04: Users can paste source code for analysis.

FR-05: Users can upload source code files.

FR-06: Users can submit GitHub repository URLs.

FR-07: The system can perform AI-powered security scanning.

FR-08: The system can detect vulnerabilities such as SQL Injection, Cross-Site Scripting (XSS), Command Injection, Authentication Flaws, Hardcoded Secrets, and insecure cryptographic implementations.

FR-09: The system can classify vulnerabilities according to severity levels (Critical, High, Medium, Low).

FR-10: The system can generate detailed vulnerability reports.

FR-11: Users can download reports in PDF, JSON, and Markdown formats.

FR-12: Users can view scan history.

FR-13: Administrators can manage user accounts.

FR-14: Administrators can monitor system activities.

FR-15: The system can generate AI-based remediation suggestions.

FR-16: The dashboard can display vulnerability statistics and risk scores.

3 Non-Functional Requirements

NFR-01: The system should load pages within 3 seconds under normal conditions.

NFR-02: The application must support responsive design for desktop, tablet, and mobile devices.

NFR-03: User passwords must be securely encrypted.

NFR-04: The system must ensure data privacy and protection.

NFR-05: The platform should support scalability.

NFR-06: The system should maintain high availability and reliability.

NFR-07: API responses should be optimized for performance.

NFR-08: The user interface should be intuitive and user-friendly.

NFR-09: Uploaded source code must remain confidential.

NFR-10: The system should support future integration with additional AI models.

4 User Roles

4.1 Guest User

Responsibilities:

- Visit the landing page.
- Learn about platform features.
- Register for an account.

4.2 Registered User

Responsibilities:

- Upload source code files.
- Paste source code.
- Submit GitHub repositories.
- Request security analysis.
- View vulnerability reports.
- Download reports.
- View scan history.
- Manage profile information.

4.3 Admin

Responsibilities:

- Manage user accounts.
- Monitor platform activities.
- Review generated reports.
- View platform statistics.
- Manage system settings.

5 Use Cases

5.1 User Registration

Actor: Guest User

Action: User submits registration information.

System Response: System validates information and creates a new account.

5.2 User Login

Actor: Registered User

Action: User enters email and password.

System Response: System validates credentials and grants access.

5.3 Upload Source Code

Actor: Registered User

Action: User uploads source code files.

System Response: System stores the files and initiates security analysis.

5.4 Submit GitHub Repository

Actor: Registered User

Action: User submits a GitHub repository URL.

System Response: System retrieves repository content and begins vulnerability scanning.

5.5 Generate Security Report

Actor: Registered User

Action: User requests analysis results.

System Response: System generates a detailed vulnerability report including severity levels, CWE references, CVSS scores, and remediation suggestions.

5.6 Download Report

Actor: Registered User

Action: User selects report export option.

System Response: System provides downloadable PDF, JSON, or Markdown report.

6 Technology Stack

6.1 Frontend

- React.js
- Next.js
- Tailwind CSS

6.2 Backend

- FastAPI (Python)

6.3 AI Integration

- Claude API
- GPT-4o API
- Gemini API
- DeepSeek API

6.4 Database

- PostgreSQL
- MongoDB

6.5 Version Control

- Git
- GitHub

7 GitHub Repository

https://github.com/0xSaikat/pluto-ai